

Component Based Architecture for an Intelligent Mobile Manipulator

Paul G. Plöger, Kai Pervözl, Christoph Mies, Patrick Eyerich, Michael Brenner,
and Bernhard Nebel

Abstract. We describe the development of an architecture for the DESIRE technology demonstrator based on principles of classical component based software engineering. The architecture is directly derived from the project requirements and resides on the concept of an *Autonomous Component* utilizing a smart feedback value called *WishLists*. This return type is able to provide expert advice about the reasons of occurring failures and give hints for possible recovery strategies. This is of key importance to advance towards robustness. The integration of an AI task planner allows the realization of higher flexibility, dependability and capability during task execution and may resolve conflicts between occurring WishLists. Furthermore the necessity of a central system-state model (*Eigenmodel*), which represents the current state and configuration of the whole system at runtime, is explained and illustrated. We conclude with some lessons learned.

1 Introduction

The goal of DESIRE was to build and program a mobile manipulator, which operates in non-laboratory, everyday and unconstrained situations amidst human users. The tasks to be accomplished are typical pick and place services or collection and

Paul G. Plöger

Bonn-Rhein-Sieg University of Applied Sciences, Sankt Augustin, Germany

e-mail: paul.ploeger@h-brs.de

Kai Pervözl · Christoph Mies

Fraunhofer IAIS, Sankt Augustin, Germany

e-mail: {kai.pervoezl, christoph.mies}@iais.fraunhofer.de

Patrick Eyerich · Michael Brenner · Bernhard Nebel

Institut für Informatik, Albert-Ludwigs-Universität Freiburg, Germany

e-mail: {eyerich, brenner, nebel}@informatik.uni-freiburg.de

delivery services in a scenario like kitchen clean-up. The contribution of the project partners ranged from pre-product components in their final review stage, right before product launch, to new algorithms tackling open research issues.

The overall robot, which is called *technology demonstrator* or plainly *system* from now on, is described in greater detail in the first article of this book chapter, see “Robot Hardware Design in the DESIRE Project”. Those hardware components -which are especially important for the development of the architecture- are listed in Fig. 1 and are repeated here for convenience. In this figure FHG IPA denotes the Fraunhofer research “Institute für Produktionstechnik und Automatisierung”, NEOBOTIX is an SME and KUKA and Siemens represent industrial partners.

component	lead partner	focus
<i>Head</i>	FHG IPA	3D modeling
<i>Platform</i>	NEOBOTIX	Mechatronics
<i>Perception</i>	Siemens	Scene Analysis
<i>MobileMa- nipulation</i>	KUKA	Flexible Con- trol

Fig. 1 Characterization of HW components, partners and research focus, for details see [7].

Please note that some components, e.g. *Perception*, are so rich in functionality that they themselves consist of a number of subcomponents. For the internal architecture the respective task force partner -not mentioned in Fig. 1- take responsibility.

In sequel we explain the main ideas underlying the development of the DESIRE control architecture. After noting the related work in section 2 we first describe our three main goals in section 3. Then in section 4 we illustrate the guidelines which are used to find functional entities, component boundaries and the communication between them. Then in section 5 it is explained how we defined the missing software glue components which are in turn explained in detail on implementation level in section 6. The next section 7 describes the planner instance in context of the overall system. Finally we close with some remarks on the lessons learned and a summary.

2 Related Work

Many details and ideas of this work can be traced back to ideas and seeds found by pioneers in the field and their earlier publications. Our main idea of execution being graph based can be traced back to RAP [2] where a strict formalization of a task network using failure and success conditions was presented first. The ideas behind this LISP-based formalism were made more amenable later by Simmons and Apfelbaum [22] in their TCA/TDL approach which embedded the hierarchical task tree and its respective success/failure condition tracking into an annotated C/C++ program.

Yet over the years the main focus for robotic software changed from understanding and modeling the workings of an autonomous vehicle correctly to practical aspects of how to combine existing software to larger working systems. This was probably

due to the fact that more and more particular working solutions for partial problems became available with a major need to orchestrate these into one common system.

CARMEN is an approach to this problem which was published in 2003 by Montemerlo [14]. Very similar to the approach presented here it tried to adhere to firmly established software construction guidelines instead of enforcing a tight structural framework on the researcher. It mainly aimed at the integration of algorithms in the field of localization and mapping and was one of the earliest approaches to couple activities only loosely. This insight was pushed even more by Player/Stage [26], which provides a client/server approach to communicate to all sensors of a robot using a very slim handmade TCP/IP protocol. Exploiting this interface a replacement of a robot by a simulator works all in a breeze and was one of the main success factor of Player/Stage. The DESIRE architecture adopts this insight on the low, raw data level for the sharing of sensors needed by more then one autonomous component. ROS [19] can be seen as a recent successor of this ideas. Based on the weak abstraction of any activity in a robot being encapsulated into a node ROS is essentially data-flow oriented. It offers very strong debugging aids which resembles our ideas of centralized monitoring services strongly.

A turn to lightweight modern software architectural principles can be observed also in Makarenko's original work on ORCA [3] and its extension on lightweight frameworks [11]. He strongly argues for a mild but better layering of existing framework in order to make open source research in robotics more prolific. According to Ingrand [12] this process can be used productively by transposition of successful software production design flows into the robotics domain. His Geno3M fosters ideas from model-driven software development and thus promises to make robotic middleware more adaptable to the current combination of given problem and given robot.

For robotic architectures it is by now a widely accepted fact that in the large a major revision of the methods and technologies currently used for the implementation of robotic software is called for. Like in many other areas the construction of robot control software should focus more on the deployment of common off-the-shelf software and components in combination with a conceptually clean integration of them. This topic has drawn major attention of a large number of researchers [4], conferences session and workshops and also some major organizations initiated and funded respective activities [18]. The issues are much under debate and far from being settled.

3 Architecture Goals

The three most distinctive goals of the DESIRE architecture may be stated (in decreasing order of priority) as follows:

1. Provide a framework for system integration framework of multi-vendor, multi-OS, multi-layered hardware components possessing own sensors and offering software interfaces, which are in no way restricted. All given hardware components have been developed independently and target vastly different user groups from robotic researchers mostly interested in open source to potential industrial customers using proprietary software.

2. DESIRE component providers have two main tasks namely component development and refinement and then their integration into the system. The goal is to make both tasks as simple as possible, but not simpler.
3. The technology demonstrator operates under everyday conditions as robustly as possible.

The ranking is an immediate consequence of the heterogeneous composition of the consortium and the large diversity of system components. Control paradigms ranged from hard real-time operating systems down to statically scheduled polling loops. Some algorithms are realized as dedicated FPGAs, others used MATLAB based matching algorithms to implement specific functionalities. Virtually no prevailing or unifying principles could be found and the heterogeneity of interfaces ranged for example from RS232 to Profi-Bus. The technological readiness level of particular components covered the range from experimental to pre-production quality.

Enhancing the functionality of their own component without scarifying too much time on integration issues was the main goal of the majority of the partners. All partners used different programming styles and environments as well as different design styles, all of which varied grossly. The gain of a distributed programming was promoted by offering shared functionalities like centralized debugging messages, service registry, system-wide consistent interpretation of status informations -especially success or failure notifications- or just as plainly as by the use of a common, centralized system clock.

Point 3. above refers to robustness which is implicitly targeted by the technology demonstrator as it is designed for operation under every-day conditions so high robustness is required. It is generally conceived that robustness mainly concerns fault-free operation of the overall system, perception and actuation. For example the perception has to work under varying lighting conditions when identifying objects. Secondly e.g. the information on the internal system health state have to be communicated, evaluated and maintained. Robustness is as much a system attribute as a component issue. So in DESIRE we took a wider view. We compensated for hardware, software and/or execution errors on *system level*. The system can react to failures by re-tries, re-scheduling using redundant components (e.g. like the left arm instead of the right) or re-planning. Since it is impossible to make precautions for all conceivable error situations the use of a symbolic planner as central part of the architecture is mandatory. In this way the system can adapted its internal world model, can keep track of changing environment conditions and can re-plan operation sequences at runtime.

As will be made clear in sequel goals 1.,2. and 3. have a pronounced impact on the choice and use of the middleware. For details see section [5](#)

4 Architectural Concepts

The architecture task-force in Fraunhofer IAIS benefited from experiences gained in other projects like the production and marketing of mobile platforms, business process modeling software based on service oriented architectures and BPLe and

other publicly funded research projects. Integrating all components into the technology demonstrator based on pure, generic ICT patterns -like those found in current SOA based architectures- proved infeasible. Non-functional requirements like timing bounds, synchronization and calling schemata and especially detailed specification of behavior in case of failure dominated the requirements for the service level agreements between the components.

The potential number of control paradigms, communication types and monitoring mechanisms is too large and too much robot and task specific and thus prohibitive to allow for a generic implementation.

Terminology: We need to define some terminology and abbreviations. All mentioned concepts are explained in much greater detail in sequel.

- *CA*: a *composite action* is a coarse grained unit of execution offered by a component. It has a well defined interface, may be called from third parties and it offers a planner compliant interface description including pre conditions, post conditions and effects. *CAs* map one-to-one to plan operators.
- *LLA*: a *CA* can fall into *low level actions (LLA)*. These are atomic, invisible for the planner, but visible for the sequencer.
- *ACo*: an *Autonomous Component (ACo)* is “a unit of composition with contractually specified interfaces and explicit context dependencies only”. An *ACo* is a robotic “component (that) can be deployed independently and is subject to composition by third parties” (based on Szyperski’s definition of a software component in [23]). An *ACo* provides some build in deliberation and some failure detection and diagnosis capabilities.
- *CCo*: a *Coordinated Component (CCo)* is a time-limited tight coupling of more than one *ACo* during execution.
- *Command*: a *Command* in the DESIRE middle-ware is a call to an *ACo*, which has a asynchronous feedback value.
- *Operation*: an *Operation* in the DESIRE middle-ware is a call to an *ACo*, which has an synchronous feedback value.

Use-Case Driven Approach: The architecture task-force started by defining and clarifying the projects demands employing several use cases. Typical scenarios were specified like tidy a children’s room, tidy a kitchen or open a door. Then these tasks were decomposed into sequences of composite actions which in turn are decomposed into low level actions. From this point on, when speaking just about actions, we refer to composite actions.

For example the *CA* “grasp an object” can be decomposed into a sequence of *LLAs* “approach object”, “determine relative pose of object”, “calculate grasp pose for hand”, “approach object with arm”, “grasp hand around object”, “lift object”. From a system point of view a *LLAs* is treated as atomic action, which is visible to the sequencer only. *LLAs* are called skills in other architectures, see [2]. *CAs* are those component actions which interface to the planner. They map one-to-one to plan operators. This is different from *LLAs*, which are invisible for the planner. Plan operators have preconditions and postconditions, which are part of the the interface descriptions of the respective composite action.

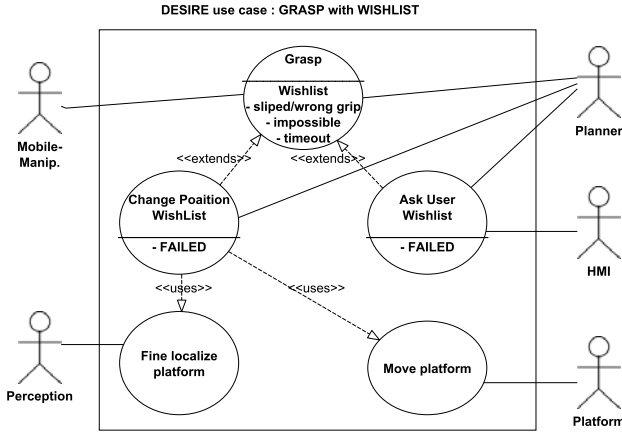


Fig. 2 The DESIRE use-case: grasping with failure resulting in a Wishlist as a smart return value

The analysis of the use-cases strongly suggested the main architectural abstraction *ACo*. The already given hardware component boundaries clearly advocated the use of component-based software engineering techniques (CBSE) to realize the respective software architecture on system level. The definition of an *ACo* matches almost all individual characteristics for each of the hardware building blocks mentioned in Fig. 1. Each of these *ACos* offer a public interface to their *CAs* and sometimes also to their *LLAs*. But the contracts may differ from one component to the next.

For example the DESIRE platform component allows to give a command “Goto pose(x, y, α) in world frame and under obstacle avoidance”. But it may not be taken for granted that an *LLA* like “put wheel A to rotational velocity v_a ” is available.

ACos may interact with their external environment self-sufficiently in feedback loops of their own and may be capable of component specific deliberative tasks like motion planning or fault diagnosis. So for some unspecified period of time they may viewed as acting independently from the rest of their hosting system.

In contrast to a classical software-only component, an *ACo* has a) usually a varying runtime for its implemented behavior, b) a low level interface either connecting to a dedicated raw sensor which is owned by the component or to some raw sensors might have a dual use in the overall system. For example the laser range finder may be subscribed by a localization and mapping service and may simultaneously be used for leg tracking of persons. Furthermore c) *ACos* may comprise real-time control loops and the definition of an *ACo* should never cut real-time loops since this would put a very high demand on the communication channels, see below under 5.

In contrast to classical robotic architectures, an *ACo* contains internally already several levels of a control architecture like e.g. in 3T. Unlike many other predecessor architectures, [1], [2], [15], [17], DESIRE components provide a high degree of self sufficiency, local knowledge, process intelligence and real-time capabilities. For

example the *Platform* component possesses already the built-in ability to navigate from point A to point B using a loadable map of its environment and while moving avoids obstacles. An independent path planner is enclosed in this *ACo*, which is invisible from the system level. The same holds true for *ACo Perception*, which maintains an object databases and object recognizers together with a scene analyzer and gesture and face recognition. The system architecture did not address these structures of finer granularity below the component level but rather delegated architectural issues to the respective component providers. The *ACos* interfaces on a higher level of abstraction to meet the demands of execution, monitoring and planning. Using *ACos* the DESIRE architecture combines classical, hierarchical hybrid robot control architecture with component oriented software architectures.

Extension of ACos: Nevertheless there are tasks whose execution demands for a loose coupling of components beyond and above the *ACo* level. For example when searching for a region of interest during object detection or when tracking a person or when recognizing a pointing gesture, the head pose needs to be adjusted continuously. This requires the coupling of several *ACos* (i.e. here *Head* and *Perception*) on system level. This can be realized only in soft real-time since communication is done using a middleware (see below [5]) which has a too large latency. So the concept of a *CCo* was defined. A *CCo* is a time-limited tighter coupling of more than one *ACo* during execution. Thus *CCo* allow to represent combined perception and control actions for the planner and sequencer as single indivisible actions, see below [6.3].

A second building principle is the avoidance of lateral coupling. This is an immediate consequence of the encapsulation of hardware units into *ACos* which almost complete obstructs any sideway communication. Coupling of *ACos* takes place only at the beginning of their control flow and/or at the end of it or at begin / end of their respective data flow. At the very top level at command reception or at bottom level when reading sensors, we can interface to each *ACo*.

Lastly to reach the robustness all errors of *LLAs* which cannot be remedied inside the respective *ACo* have to be propagated onto system level. Thus sequencing of complex tasks into actions or the retrials of individual failing actions has to be done on system level as well as execution monitoring. A planner which is tightly coupled to a sequencer is essential for system integration. During fault detection the planner closely cooperates with the sequencer. Thus both need some tightly coupled shared memory. Furthermore a global state history to avoid respective plans is needed. An "Eigenmodel" (EM) as common infrastructure was added as a separate *ACo*. It provides centralized logging channels for record tacking and/or debugging.

Communication: A closer analysis of the type of activities triggered by the sequencer and executed by the respective *ACo* revealed that the necessary communication is governed by just two message patterns. On one hand we find some longer running, asynchronous calls mostly when performing *CAs* like "grasp an object" with delayed return values. On the other hand we have short synchronous calls with immediate return values. The first communication pattern we called *Commands* and the second *Operations*. Either type can be generated from a code template. We tried

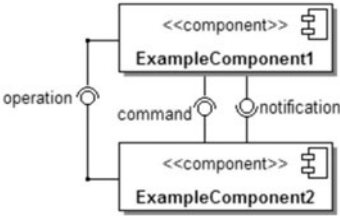


Fig. 3 Generic communication pattern: *Operations* have an immediate return value while *Commands* are asynchronous and will send a *Notification* after termination

component	lead partner	topics	OS
<i>HRI</i>	Univ. Bielefeld	Language understanding	Linux
<i>Planner</i>	ALU	Ontologies	Linux
<i>AS</i>	FHG IAIS	Architectures	Linux
<i>EM</i>	and ALU	Self Diagnosis	Linux
<i>Head</i>	FHG IPA	3D modeling	Wind.
<i>Platform</i>	NEOBOTIX	Mechatronics	Linux
<i>Perception</i>	Siemens	Scene Analysis	Wind., Linux
<i>MobileManipulation</i>	KUKA	Flexible Control	Vx-Works, Linux

Fig. 4 Added software components, partners and operating systems. For hardware dominated components see Figure 1

to base the construction of the system architecture on this principle of reuse of pattern as much as possible.

Finally to ease the actual task execution we treated movement as modal commands, which means the technology demonstrator never drives and manipulates at the same time.

5 Design

As already indicated above there was a need to add further software-only components which complement the hardware dominated *ACos*. Especially the system needs a Human-Robot-Interaction (*HRI*) component and system level task coordination including failure detection. Figure 4 lists these additional software *ACos* concretely the *HRI*, the *Planner*, the sequencer called *Ablaufsteuerung* (*AS*) and the “Eigenmodel” (*EM*).

5.1 The Planner

The base line for the planner is a modified extended version of the well known FF planner [10] using metric fluents and external modules. A novelty lies in the interaction of the planner with the *HRI*. Both cooperate to resolve a spoken utterance into a planner compatible goal. For example a human usually uses imperative commands (e.g., “Take a plate!”), while a planner needs a logical formula representing the goal state e.g., $\exists x : (plate(x) \wedge holding(x))$. The planner generates its goals from the natural language input processed in the *HRI* component. Roughly speaking, the planner searches for some key words in the output which are bound to actions in


```

/**
 * moves platform to specified pose
 * in global reference frame
 *
 * @precondition platform moveable
 * @precondition map loaded
 * @precondition platform localized
 * @precondition pose reachable
 * (path planner feedback)
 * @effect getCurrentPose(out Pose
 *   curPose); curPose == pose
 *
 * @param pose in: absolute pose
 * @return Feedback
 */
desire::Feedback
  moveAbs(in desireDT::Pose pose);

```

Fig. 5 Example for the planner compliant representation of the CA “move-abs” of the *Platform* component.

```

module feedbackDataTypes {
  enum OperationStatus {
    Running, Finished, ProgrammingError,
    HardwareError, GoalNotReachable
  };
  struct Time;
  struct Feedback {
    short komponentenID;
    long operationID;
    OperationStatus status;
    string statusDescription;
    Time componentTimeStamp;
    Time creationTime;
    Time durationToFinish;
  };
}

```

Fig. 6 Data type Feedback message

the planning domain and generates the goal condition out of the effects of these actions. During the generation process, the planner detects individual object instances among the objects referred to in the instruction (e.g., within the instruction “Give me the plate!” the user rather refers to one distinguished plate than to an arbitrary one). Then it plans separately for all combinations of objects currently in the abstract world model which fit to the type constraints of the distinguished objects. In that way, the planner is able to discover some forms of ambiguity. If such an ambiguity is detected, the planner informs the sequencer, which in turn tries to dissolve the ambiguity, e.g., by checking if there is a user pointing gesture to some of the objects.

5.2 The Sequencer (*Ablaufsteuerung AS*)

The sequencer component AS has a classical design. The general idea follows a modified observer pattern which is implemented using bidirectional RPC provided by a CORBA based middleware, see below. The sequencer issues commands to the components by a call to one of the CAs offered by the respective component. Recall that *Commands* are defined as non-blocking remote procedure calls, the handling of the triggered activity is in lockstep to the sequence of incoming calls. Every calling component has a unique component identity and every command issued by the caller is awarded a per-call-identity ID which uniquely tags it. This command ID is generated by the caller. An issued command is acknowledged with an immediately return feedback message. This message and all subsequent feedbacks indicate the execution status of the issued command, like e.g. in-liquidation, termination or

error. All commands, which have a preconditions and an effects part (see Figure 6) comprise the planning interface of this *ACo*. The maximum expected time to return a possibly delayed feedback notification is known beforehand. Thus reaching Timeout is an indication of an error in communication.

The retrieval of information from all components uses “get” commands. Information not provided in the feedback message will be delivered via an out parameter of a separate get-command call.

The sequencer *AS* can ask for the processing status of a command. A component reports all significant events to the *AS*. To this end *AS* implements a notification function. Significant events are the unexpected termination of a command execution or the successful completion of a command execution.

5.3 The Eigenmodel

One consequence of the concept of *AS* is that the individual components are not aware about the other components of the host system. Since many tasks require cooperation between different components, a centralized information center is necessary. This is called *EM* and provides several services.

For the *ACos*, the most important service is the mechanical configuration of the robot system, in which they have been embedded. This information is required for example by the perception component, which provides the service of detecting the pose of objects in world coordinates. To achieve this, the *EM* provides the pose of the camera in the world frame, thus the perception component does not need information about the current robot configuration. There is no difference for the perception if it is working on a wheeled robot with a fixed mounted camera or on a humanoid robot with several dozens degrees of freedom. The mechanical configuration of the robot is modeled as a kinematic chain, which is composed of several nodes, connected via links. Nodes can be rotational or translational joints but also static nodes. The position of nodes can be set and read from external components.

Furthermore the *EM* serves the purpose to collect runtime state information of all components in one central storage and to make them available on system level. It may store the generalized coordinates of each hinge. Finally the *EM* offers logging, archiving and lookup services.

5.4 The Middleware

For the choice of an appropriate middleware several open solutions were compared with solutions providing at least an interface definition language (IDL). This is an decisive feature since it substantially eases the marshalling and de-marshalling of the very different data structures being exchanged between the respective components. The first group contained MiRPA, MCA and Player, while in the second we investigated WSDL based solutions especially CORBA and ICE distributed by ZeroC [8]. All candidates were checked against the requirements. Most solutions in group one lacked support for at least one of the used operating systems. Since

a mapping between WSDL and other middleware, which provide an IDL, is easily possible the decision was narrowed down to either CORBA or ICE.

ICE is a recent and lightweight reimplementation of the main CORBA concepts. It has a very steep learning curve and is much better performing than the original. Yet there is significant downside: ICE is single source and not standardized, commercial licenses are quite expensive and again there is no support for VxWorks, which was needed by one partner.

On the other hand CORBA is complicated and although standardized some services are only implemented specific to a vendor. Yet CORBA gains in stability, it is multi source has well established support channels, offers support for all operating system. Finally TAO is a high quality real-time enabled open source implementation of CORBA. Thus TAO/CORBA was chosen as the appropriate middleware.

To ease the learning curve only minimal CORBA services (especially naming service and brokerage) were used in an IDL based implementation template for a standard component communication interface. Component interactions use only three message types namely operations, commands and notifications. This interface proved to be as simple as possible, but not simpler, for the purpose of system wide communication. The interface specification in CORBA IDL for all components and all partners resulted in only about 2000 lines of code being implemented. To represent the services offered by an *ACo* in a way which is compatible to the needs of the planner there is a generic form to be filled in by the component provider. This form contains a table describing component attributes and a description of functionalities offered.

The information needed by the planner contains preconditions and expected effects. It is added to the interface description of all components in the form of a comment with a particular format, see Figure 5. The aforementioned 2000 IDL lines included all preconditions and effects.

6 Implementation Details

In this section we describe some implementation issues and how special aspects of the DESIRE architecture were realized. This closer look includes the *AS*, *WishLists*, *CCs*, and the *EM*.

6.1 The Sequencer Implementation

The *AS* component is a sequencer and scheduler. The sequencer works as a central mediator of requests which in the end leads to calls of *CA* offered by the respective component interfaces. The components themselves can keep partial states of their own and can implement own task control instances internally. For example, it is possible that a component starts to serve an ordered command execution of a *CA*, however, only partly executes contained *LLAs* and resumes to the original *CA* at a later time of the control sequence. This can be seen in Figure 7. The top layer of nodes is a sequence produced by the *Planner*. The *AS* interprets the plan and issues

It should also be noted that the *AS* breaks in some sense the autonomy of the *ACos* since it may have internal knowledge about the working of the the respective *LLAs* making up a *CA*. This concept is well known from gray box integration which is employed in this case. The *AS* does not break the *CA* into *LLAs* itself rather it knows and models how the *ACo* will break the respective *CA* into *LLAs* by a fixed recipe. For example in step 3. of the **Example Scenario** below (i.e. grasp the cup) the *CA* “grasp” falls into *LLAs* “reach pre-grasp pose”, “calculate the grasp”, “reach out and grasp” and “move arm to transport pose”. This fixed sequence is known to the *AS* but unknown to the planner. Any of this *LLAs* may fail and may signal a *WishList* back to *AS* so it can take care of error recovery early and on top of the *ACo*.

For example to request the move of the platform in the global frame the IDL code from Figure 5 is used. Special knowledge about conditions and the environment is encapsulated in the components thus in a modular way. A successful completion of an action causes the control flow to leave the tree node on top level. It induces a state transition, which is transparently propagated to the planner component.

If an action is not successful, the failing component generates an abstract advice on how to alter the system state, which then, as far as diagnosable, can fix the error. This special return values are of data type *WishList*. They are reported to the *AS* and are further processed by it. *AS* may generate an own response to handle a *WishList* or might trigger a re-planning with a *WishList* as a new initial constraint.

The *AS* can track the command processing by calling the function “getCommand-Status()” with the ID of the command as a parameter. Detailed information may be obtained by additional “get” commands. The *AS* can also affect the execution of a command, for example by

- abortion (for example, in case of a global platform break)
- interruption (pause)
- issuing of new parameters (set command)
- restarting

This process control may be offered by a component or not since it is not mandatory.

To illustrate the working of *AS* take the following, very easy, yet typical, scenario.

Example Scenario

1. The robot should drive to a certain table (being in the same room),
2. recognize a certain cup on it and, finally,
3. grasp the cup.

This example requires several actions to be coordinated. Note that 1., 2. and 3. are simple tasks, which are quite well understood and independent from one another. Thus, they can be implemented independently. Yet the complete task can only be solved by a suitable orchestration of all participating components.

Each action can be performed by a designated component, so these have to be coordinated. In the Example Scenario above the component *Perception* performs

the recognition task, *Head* moves the head and thereby the camera, *Platform* drives the undercarriage and *MobileManipulation* controls the arm and hand unit. The first subtask requires a collaboration of *Perception*, *Head* and *Platform*, the second one is performed by *Perception* and the last task needs the *Perception*, *Head* and *Mobile-Manipulation*. The example shows the principle working of AS as interplay between different components, see also Figure 7.

- The planner generates a sequence of plan operators, which is interpreted by AS as a graph T . AS executes this graph by generating calls to some basic services in the *Perception ACo* to localize the table.
- A subcomponent of *Perception* named *SceneAnalysis* checks if the information can be generated with sufficient precision using the current scene model. If yes, it will return the position as a result. If not, an expert advice module will generate a *WishList* like: change camera position or change robot position.
- From this *WishList*, see below 6.2 AS generates a new sub-graph T' of T , in necessary cases also using the planner.
- After T' is finished, AS continues with T .

Several graphs may be processed in parallel, i.e. new orders are accepted and processing can be started before an initial task has been completed. The example also shows that *ACos* are no mere recipient of orders of AS but may also produce new tasks dynamically for AS while interacting. Although there is a complete world model in the *SceneAnalysis* sub-component, other *ACos* keep localized knowledge about states and environments. Thus the burden to maintain an omniscient world model is lowered by this decentralized approach.

Besides call interaction and *WishList* reception from actuation or perception components the AS is able to execute plans that can be formalized as directed graph with a root node and exactly one leaf node denoting the execution finish. Each node in such an execution graph is a call to an *CA*. An action can either be a simple call to a component or it can recursively be an execution graph itself. Thus, it is simple to build hierarchical execution graphs, which is an elegant way to raise the granularity to facilitate planning. Failure recovery is performed in each action. Thus, an action can try to recover from a failure and the whole graph is further executed. If a local recovery is not possible, the action fails and the next higher graph has to recover from the failure. This can recursively happen until a top level action fails and the whole plan does. In these cases the AS triggers the planner to re-plan on the current goals and, if that also does not lead to a successful execution, the user must be asked in order to help with failure recovery.

The AS can handle dynamically changes of execution graphs, additionally, hard-coded plans are used to perform regular action sequences. For example, the task of plan generation and parsing is defined in an execution graph. Thus, the AS can execute plans, but also contains plans in order to perform regular fixed tasks. Important is also its ability to recover from execution failures. This later can be done in every layer of the execution graph, since a *WishList*-hint for recovery is given by the actions.

6.2 WishList

Basically a WishList is an abstract data type used to give AS a hint on how a failed task can still be accomplished. The implementation of *WishLists* has been phrased in a special language to enable components to communicate to the whole system but to avoid a dependency on other components.

Considering the Example Scenario in section 6.1. A useful *WishList* from the *Perception* might be a repositioning of used sensors. If the robot receives the task of exploring a table and only parts of it are visible from the current camera position, *Perception* can send the wish of repositioning the camera to AS, which decides if the wish can be realized. If so, the AS initiates a repositioning of the sensor, to allow a changed perception and thus a change in the world model.

As already stated, the *WishLists* are a possibility for the components to inform AS how the system performance may be optimized. This is especially useful in case of failures. An *ACo* itself has expert knowledge about a possible cause of its own failure and can suggest a change of the world state in order to reach the current subgoal.

There may occur several wishes from different *ACos* at the same time. Thus, the AS has to “reason” about how to fulfill these wishes. No component can rely on the fulfillment of its wish, thus the control of the whole system still is executed by the AS. If *WishLists* are not contradicting, several wishes can be fulfilled in parallel. Basically, *WishLists* can occur nested, another *WishList* can occur while the current wish is fulfilled, see Figure 7 right side. All can be sequential and in parallel as well as contradicting or not. Thus, the scheduling of these *WishLists* must also consider the current resource allocation in the system.

Several kinds of *WishLists* have been predefined. Since the *WishList* usually is used in the context of failure recovery, the language was defined to be (1) easily interpretable by computers and (2) readable by humans in order to keep track of the robot’s activities and for debugging. Usually it is difficult to map such formalisms to sentences in a natural language. Yet for a *WishList* it is possible in principle to rephrase it in natural language.

Each *WishList* has the following form: `<type> <caller> <args>`, where `<type>` refers to the type of the *WishList*, `<caller>` denotes the component that sends the wish and `<args>` denotes the message body that depends on the *WishList*’s type. It is important that the concept simultaneously supports quantitative and qualitative wishes. Examples are like `CHANGE_POSITION manipulation POSE 10.8 20.4 30.2 40.3 50.9 60`, where the component manipulation wants the platform to be moved to a certain pose defined in terms of real valued numbers. Or another instance of the same *WishList* type is `CHANGE_POSITION manipulation CONTEXT grasp OBJECT 4`, where the manipulation wants to be positioned such that it can grasp the object 4. Thus, the context `grasp` is a qualitative constraint representing the semantics that the new position must be reachable by the platform and object 4 can be gripped from that position, too. The resolution of what poses allow for grasping is not resolved by the AS but it is delegated back to an internal grasp planer which is offered by *ACo Manipulation* as service.

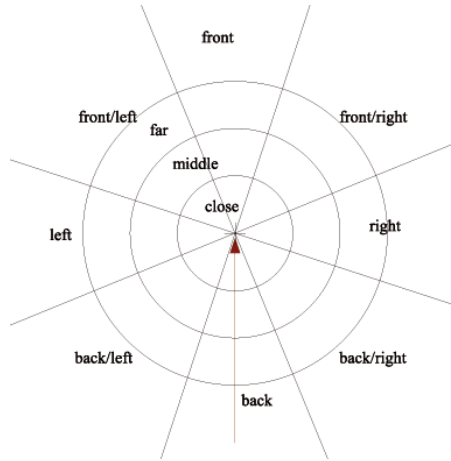


Fig. 8 The qualitative description used in the *WishList* data type. [21]

To resolve qualitative to quantitative wishes, a qualitative calculus has been used [21] in order to formulate relative position descriptions. The target position is defined relative to a reference object and both contained in a coordinate reference. The target position is the position to be described, the reference object occurs in the *WishList* and the coordinate reference is grounded at certain point of the robot (e.g. center of gravity) that is also a part of the *WishList*. Distances are denoted by the three qualitative constants *close*, *middle* and *far*. Quantitative distances are assigned to these constants, which then define concentric, ring-shaped zones around the reference object. Orientations are defined by the constants *front*, *back*, *left*, *right*, *front_left*, *front_right*, *back_left* and *back_right*, where each defines a sector of 45. A qualitative pose is then a tuple of orientation and distance. In Figure 8 every cell denotes exactly one qualitative pose. The red arrow denotes the directed straight from the coordinate reference to the target position. Thus, it also denotes the orientation 0 represented by *front*. The specification of this world model is derived from Schiffer et al. [21]. `CHANGE_POSITION manipulation PART arm OBJECT 4 POSE close, back` is an example of such a qualitative *WishList*. The component `manipulation` wants the robot part `arm` to reach the qualitative pose *close, back* relative to the object 4, i.e. the arm should be positioned just behind the object.

All these qualitative terms are relative to the mobile manipulators' point of view and relative to the object to be grasped or aimed for. They can be used by reasoners able to deal with positions. Furthermore they can be resolved to numeric values by services available from the respective *ACo*. In our scenarios this turned out not to be necessary and thus we used a fixed mapping.

In order to schedule parallel *WishLists* correctly, a resource management for the architecture has been specified and can be resolved by the planner. Assuming that data, e.g. camera pictures, can be accessed simultaneously by several components,

only the control of certain devices or software services is managed. Therefore, each service existing in the system defines its needed resources. If the resources are located in the hosting component, there exists no further problem because the host has a priority access to all its devices. Hosting components are specified in the configuration of the robot and can be looked up in the *EigenModel*. For example the SICK laser range finder is hosted by the Navigation *ACo* and may also be used by the people detection services.

But some constraints concerning the applicability of services are component overlapping. For example, the manipulation needs the platform to stay at its current position during gripping. Pseudo-resources have been introduced to be able to model such dependencies since conflict resolution is handled via a resource manager.

In general hardware components can demand that the origin of their coordinate frame must not be changed while they are operating. These origins are called *mount points*. They are treated as pseudo resources, so if they are in use access can be blocked. E.g. if a used *mount point* was tagged by the manipulation component platform relocation cannot take place although the manipulation component itself will actually not need the drive unit. i.e. access is blocked via the used *mount points*.

A resource request is formulated as follows. A request is either a petition for *full-control* stating that no other service may actively control the resource, or for *no-change* stating that no other service may change the current control state of the resource. The assignment of one resource is straight-forward. If nobody uses the resource, it grants the next request. If *full-control* is requested, it is only granted if no other service is currently using the resource. If *no-change* is requested, it is granted if nobody uses the resource exclusively.

The *no-change* petition has been introduced in order to minimize the restrictions, because often the components require that nobody changes the resources' state and an exclusive usage would forbid a simultaneous usage.

All in all the resources can simply be represented within each of their hosting components. Only the overlapping mount point need to be defined system wide. The advantage of this is that the encoding of the resources need not to be shared among the components. Deadlocks will be resolved priority based.

6.3 Coordinated Components (CC)

In the standard configuration, all system components are loosely coupled via the central AS component. Recall the above example scenario in section 6.1 of grasping an object from a table, it might be useful to supervise the grasp action by *Visual Servoing*. Therefore, the *Perception* and the *Head* components need temporarily to be coupled more tightly. For continuously observing, if the object is grasped correctly, the services *getObjectPose* and *trackWorldPoint* are called alternating as fast as possible. The first service provides the pose of the cup in world coordinates and the second service tracks this position with the camera to provide always an optimal view. The DESIRE architecture allows such intermediate close coupling of different components by the concept of CCs.

If a *CCs* is created, it is represented in the system like any other component and its interface is specified in IDL. The advantage of such a component is that it tightly couples the controller loops of different components, which enhances the performance and resource usage of the resulting component in comparison to the independent call usage of the involved services. *CCs* allow the realization of complex services by the combination of independent components without a-priori demanding for a close cooperation between the component providers. For their realization it is allowed to bypass the standard communication middleware of the architectural concept. Regarding the resource management, coordinated components behave exactly like ordinary components. The resource requirements are defined by the used services and will be occupied as long as the coordinated component exists and not as long as the related service is active.

The first implementation of the concept of *CCs* is based on a simple input/output coupling of the services. For all involved services, the input as well as the output parameters are defined in a XML file and each service is bound to a specific TCP/IP port. Additionally, local memory is provided to store intermediate results. The concept of a *processor* is used to extract needed data from return values of coupled services. The internal structure of a *CC* is script-able in XML and is in principle an execution graph which can contain specialized nodes for conditions and loops.

It should be noted that only certain dedicated *ACos* may be amenable to be combined into a joint *CC*. If the participating *ACos* require a high communication bandwidth the middleware will become a bottleneck. In this case the parts need e.g. to be colocated on one common multicore processor in order to have a chance to shortcut the communication and to make it essentially shared memory. Alternatively process migration can be applied yet the strong hardware dependency of the provided functionalities rendered this impossible for the DESIRE architecture.

6.4 *Eigenmodel (EM)*

The main service of the *Eigenmodel* is the provision of a centralized monitor of system states and -variables. This monitoring is necessary for failure identification and -handling. In such a complex multi-component system, many faults occur at component boundaries or involve multiple components. These faults may not be detectable on component level. Thus central monitoring service provides all system information for reasoning about the current state of the whole system. Furthermore, the *EM* provides an *Alive Management* service, which supports the coordination between several components by providing state information about all connected components. *EM* is realized as an *ACo* of its own which is integrated via the middleware and whose interface is specified as usual in an IDL file. All system components are allowed to address the *EM* directly for sending or receiving information. In the following paragraphs, the implementation is described in more detail.

The kinematic of the mobile manipulator is modeled by the usual chain of frames from world coordinate system origin over the omni-directional platform frame to the finger joints of the hands or the cameras of the sensor head. In this model, three

different types of nodes exist, namely rotational joints, translational joints and static nodes. All information about the nodes, the links and their relations to each other are provided in a XML file which is imported at system's start time. By analyzing this XML file, all system components can receive all required information about the kinematics of the system on-line. This makes it easy to adopt the whole system to a new kinematic configuration. At system runtime, all actuator components automatically send changes of joint values or node poses to the *EM* by using a logging mechanism, described below. For every new node value, the kinematic chain is updated and the new values are available system wide.

The values of the *EM* are updated continuously while the world model in the perception component is only updated on demand. Thus the two world views may diverge and a synchronization has to be explicitly called for.

The following example illustrates the usage of the robot kinematics in the system context. Involved components are the *Perception*, *Platform*, *Head* and *Mobile-Manipulation*. At system runtime, the platform control is sending the robot pose (in world coordinates) continuously to the *EM*. Simultaneously, the head control is sending the current joint values of its head pan-tilt unit. A camera system, mounted on the pan-tilt unit, provides the data for the perception's object detection. If an object is detected in the camera data, the perception component requests the pose of the camera (modeled as a node in the kinematic chain) from *EM*. By using this camera pose, the pose of the detected object can be calculated in world coordinates by the perception component without information about the transformation from the drive unit to the camera. This object pose can then directly be given to the manipulation for grasping this object.

In a complex mobile robot with many components, a large amount of data is generated continuously. Assuming a distributed system with several processors, connected via a bus system with limited bandwidth, it is not possible to store all these data in a centralized component. But depending on the current system state and the ongoing action, different component data are required centralized for analyzing the correct execution of actions. To solve this challenge, a logging mechanism is developed which is configurable at system runtime. This mechanism is based on the `log4cpp` library [5]. So called appenders are defining the destination of standard log messages. These appenders can be bound to the logging service at system runtime which allows a very easy adaption of the logging mechanism. For the *DESIRE* architecture, two additional appenders are defined. The first one logs data directly to the *EM* and may be used for receiving the poses of the actuator components. The second appender is a ring-buffer which stores the values locally at the components for a specific period of time. This is useful for example in case of the occurrence of a failure. For diagnosing the failure's origin, data from earlier time points can be utilized, even if they were not stored in the *EM* at this time. The developer of a component does not need to know which data should be logged to one or the other appender at which time. Every log message will be logged with the same method call and the current configuration of the logging mechanism defines the destination

of this message. The configuration of appenders will be done by AS. In addition to the DESIRE specific appenders, a developer can also use the standard log4cpp appenders like displaying the log messages at the console.

7 Planning

This section will describe only the architecture specific aspects of the chosen planner. For a more detailed account see the chapter on “Task planning and execution control” in this book.

To incorporate a planning system, a robot has to provide basically three kinds of information: A description of the abilities of the robot (the *actions* it is able to perform), knowledge about the *current world state*, and some adequate representation of the desired *goal state*.

It is assumed that the robot is not able to learn new behaviors on-line, so the set of possible actions is treated as static during runtime. The prevalent planning language, the *Planning Domain Definition Language PDDL* [6], is used as representation language for the definition of actions. Thereby, each action consists of a precondition, stating what has to be satisfied in a given world state in order to make the action applicable, and of effects, stating in which way the world changes after the application of the action. However, to enable the use of automatic ontology reasoning procedures such as Fact++ [25] and convenient modeling tools such as Protégé [16], PDDL is not used directly for the purpose of stating domain details such as type hierarchies and certain properties of types. Instead, the Web Ontology Language OWL [13] is used for that purpose and an automatic translation of the parts necessary for planning to PDDL is performed. Additionally, a method to integrate PDDL action descriptions and OWL ontologies which is able to reason about subsumption relations between actions has been developed.

The distributed architecture of the whole system is strongly reflected in the way the planner perceives the current world state: Information from different components has to be collected and integrated into one coherent abstract state. To achieve this, each component providing information necessary for the planner uses a proxy to transfer its relevant knowledge. The planner then collects all this information and generates an abstract world state out of it.

While a human usually uses imperative commands (e.g., “Take a plate!”), a planner needs a logical formula representing the goal state (e.g., $\exists x(plate(x) \wedge holding(x))$). The planner generates its goals out of the output of the natural language processing tool from the *HRI* component. Roughly speaking, the planner searches for some key words in the output which are bound to actions in the planning domain and generates the goal condition out of the effects of these actions. During the generation process, the planner detects determinate objects among the objects referred to in the instruction (e.g., within the instruction “Give me *the* plate!” the user rather refers to one distinguished plate than to an arbitrary one). Then it plans separately for all combinations of objects currently in the abstract world model which fit to the type constraints of the distinguished objects. In that way, the planner is able

to discover some forms of ambiguity. If such an ambiguity is detected, the planner informs the AS, which in turn tries to dissolve the ambiguity by checking if there is a user pointing gesture to some of the objects.

The base planning architecture is a slightly modified version of *FF-Module* [24], which in turn extends the well known FF [9] planner by metric fluents and external modules. Especially external modules provide great potential for planning in a robotic context, since they allow to source out time intensive computations to external instances where they can be handled in a much more efficient way.

8 Lessons Learned

The DESIRE project as whole is very ambitious, and so is its respective architecture. Providing a framework for integration while developing new software algorithms and while building new hardware all at the same time is difficult. At the same time the project reached out for goals like usability in the everyday and robustness towards faults. Furthermore dealing with over a dozen partners and their very diverse backgrounds put DESIRE under a high risk of failure. To address this the final goal remained a moving target till very late. Accordingly the concluding final demonstration was chosen neither overly complex nor too particular and the demonstrated scenarios reached from three dimensional obstacle avoidance over the interpretation of a scene with more than 100 known objects up to face identification and pointing gesture recognition. This division decoupled the demonstrated functionalities yet it did not provide one overall unifying goal for all partners. A realistic, reachable, unanimous and unifying final goal was missing.

The architecture task force was active from the very beginning and the integration was an issue up front. This helped to diminish substantially the number of integration meetings, but the parallel development of architecture while having simultaneous component development proved to be a big burden. Some architectural details were still under implementation while the deployment by some partners was already going on. It was impossible to define a freeze day since the changing requirements of components providers overlapped to much with the architectural implementation.

During development, the establishment and compulsory use of standards was grossly underestimated. Obviously this must be applied during the specification and development of interfaces or the interpretation of the semantics of data. But also the software design process may benefit from standards as was first demonstrated by the “Gang of Five” when introducing their pattern based software design style. The unified use of UML as project wide specification and description language remedies the majority of human communication failures. Although certain best practices from software engineering -like shared code repositories, nightly regression test and test-case based design- had been used at the end, an earlier establishment is called for. In case of such a heterogeneous group of programmers it pays off to invest some time and effort into the provision of a jointly mastered idiom like the use of use-cases or UML sequence- and activity-diagrams.

Finally the technology demonstrator was available as an integrated system only very late. When being on-site the prototypical character of the system impeded integration tests quite a bit. These were mainly caused by unavoidable technicalities like calibration of optical systems, network setup for the distributed computing nodes especially firewall issues when setting up CORBA services. In this situation using a system simulator as a substitute for the whole robot is a viable alternative. In fact, most of the project partners had their own proprietary simulators that were not integrated or could not be integrated. So for a substantial time span during project duration no cross-component simulator was available. A faithful system simulator providing one-to-one code compatibility was tackled only during the extension in year four, which was almost too late to unfold its potential productivity enhancements fully. This omission was especially critical for an overall test of the architecture on long running iterated goals like clearing a kitchen or alike. Using a physical simulator decouples hardware and software developments as much as possible. Models are still needed, yet the development of them can be scaled to the current needs of the system designer. Rough, coarse grained models may be programmed very fast, and even more exact physical models can usually be produced much faster than an integrated hardware prototype.

9 Summary

We outlined the research goals, approaches and solutions to define an robotic architecture for the technology platform build in DESIRE. We achieved the successful integration of disparate hardware units into a distributed architecture using *ACos* on high abstraction level. It was made possible by identifying necessary extensions on the software side, where data types like *WishLists* or components like the *Eigenmodel* offer system-wide services like health and failure monitoring and alive management to yield robustness. On component level we reported about significant progress due to recent advancements in AI planning. A thorough and comprehensive benchmarking of all components and their interplay is under way right now. Project progress and debugging worked best when all partners collaborated in person right in front of the technology platform, which took place in many common workshops. To sustain a high testing efficiency we finally also introduced a remote testing strategy using VPN, see [20]. Another substantial productivity push became possible by a reasonably exact simulation model of the whole system. Using this simulation based approach upfront should be one of the key insight kept in mind during the realization of any successor architecture.

Acknowledgements. This publication was funded as part of the research project DESIRE by the German Federal Ministry of Education and Research (BMBF) under grant no. 01IME01-IAIS and 01IME01-ALU. The authors would like to thank the anonymous referees for their numerous suggestions for improvements.

References

1. Alami, R., Chatila, R., Fleury, S., Ghallab, M., Ingrand, F.: An architecture for autonomy. *International Journal of Robotics Research* 17(4), 315–337 (1998)
2. Peter Bonasso, R., Kortenkamp, D., Miller, D.P., Slack, M.: Experiences with an architecture for intelligent, reactive agents. *Journal of Experimental and Theoretical Artificial Intelligence JETAI* 9 (1997)
3. Brooks, A., Kaupp, T., Makarenko, A., Williams, S., Orebäck, A.: Towards component-based robotics. In: *Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 163–168 (August 2005)
4. Brugali, D. (ed.): *Software Engineering for Experimental Robotics*. STAR, vol. 30. Springer, Heidelberg (2007)
5. Mo, B.: Logging and tracing in c++ simplified (August 2001),
<http://developers.sun.com/solaris/articles/logging.html>
6. Fox, M., Long, D.: PDDL2.1: An extension to PDDL for expressing temporal planning domains. *JAIR* 20, 61–124 (2003)
7. GPS Gmbh. Desire web side,
<http://www.service-robotik-initiative.de/>
8. Henning, M.: A new approach to object-oriented middleware. *IEEE Internet Computing* 8, 66–75 (2004)
9. Hoffmann, J., Nebel, B.: The FF planning system: Fast plan generation through heuristic search. *JAIR* 14, 253–302 (2001)
10. Hoffmann, J., Nebel, B.: The ff planning system: Fast plan generation through heuristic search. *Journal of Artificial Intelligence Research* 14, 2001 (2001)
11. Makarenko, A., Brooks, A., Kaupp, T.: On the Benefits of Making Robotic Software Frameworks Thin. In: Prassler, E., Nilsson, K., Shakhimardanov, A. (eds.) *IEEE/RSJ International Conference on Intelligent Robots and Systems, Workshop on Measures and Procedures for the Evaluation of Robot Architectures and Middleware* (November 2007)
12. Mallet, A., Pasteur, C., Herrb, M., Lemaignan, S., Ingrand, F.: GenoM3: Building middleware-independent robotic components. In: *Proceedings of IEEE International Conference on Robotics and Automation* (2010)
13. McGuinness, D.L., van Harmelen, F.: OWL web ontology language overview. W3C Recommendation (2004),
<http://www.w3.org/TR/2004/REC-owl-features-20040210/>
14. Montemerlo, M., Roy, N., Thrun, S.: Perspectives on standardization in mobile robot programming: The carnegie mellon navigation (CARMEN) toolkit. In: *Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems* (2003)
15. Nesnas, I.A.D., Wright, A., Bajracharya, M., Simmons, R., Estlin, T.: Claraty challenges of developing interoperable robotic software. In: *Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems, Las Vegas, Nevada* (2003)
16. Noy, N.F., Sintek, M., Decker, S., Crubezy, M., Ferguson, R.W., Musen, M.A.: Creating semantic web contents with Protege-2000. *IEEE Intelligent Systems* 2(16), 60–71 (2001)
17. United States Department of Defense. Joint architecture for unmanned systems,
<http://www.jauswg.org> (last accessed February 2007)
18. Prassler, E., Shakhimardanov, A.: RoSta - Robot Standards and Reference Architectures,
<http://www.robot-standards.eu/>
19. Quigley, M., Conley, K., Gerkey, B.P., Faust, J., Foote, T., Leibs, J., Wheeler, R., Ng, A.Y.: Ros: an open-source robot operating system. In: *Proceedings of IEEE International Conference on Robotics and Automation, Workshop on Open Source Software* (2009)

20. Reiser, U., Mies, C., Plagemann, C.: Verteilte Software-Entwicklung in der Robotik - ein Integrations- und Testframework. In: Robotik (May 2008)
21. Schiffer, S., Ferrein, A., Lakemeyer, G.: Qualitative world models for soccer robots. In: Wölfl, S., Mossakowski, T.(eds.) Qualitative Constraint Calculi: Application and Integration, Workshop at KI 2006, pp. 3–14 (2006)
22. Simmons, R., Apfelbaum, D.: A task description language for robot control. In: Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems (October 1998)
23. Szyperski, C., Gruntz, D., Murer, S.: Component Software - Beyond Object-Oriented Programming, 2nd edn. Addison-Wesley Longman, Amsterdam (2002), ISBN-10: 0201745720, ISBN-13: 978-0201745726
24. Trüg, S.: An integration of manipulation and action planning. Master's thesis, Albert-Ludwigs-Universität Freiburg (2006)
25. Tsarkov, D., Horrocks, I.: **faCT++** Description Logic Reasoner: System Description. In: Furbach, U., Shankar, N. (eds.) IJCAR 2006. LNCS (LNAI), vol. 4130, pp. 292–297. Springer, Heidelberg (2006)
26. Vaughan, R.T., Gerkey, B.P., Howard, A.: On device abstractions for portable, reusable robot code. In: Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems, pp. 2421–2427 (2003)